Software Documentation

1. Software Overview

The arcade system software transforms a 3×3 grid of physical floor tiles into an immersive, full-body gaming platform. The software coordinates four major subsystems working in real-time: floor tile monitoring and LED control, overhead visual effects (disco lighting, stepper motors, lasers), game logic processing (DDR, Memory, Whack-a-Mole, Tic-Tac-Toe), and a central communication bridge that unifies all components. When a player steps on a tile, the software detects the input, executes game rules in the browser, updates floor LEDs with color-coded feedback, and simultaneously triggers synchronized ceiling effects including dynamic light patterns and motor choreography. The system uses HTTP polling architecture where a laptop-based bridge server continuously queries both ESP32 microcontrollers , merges button states from 9 tiles and 2 IR sensors into a unified 11-element array, and routes commands bidirectionally between the web-based game interface and embedded hardware. This creates a seamless experience where digital game logic controls physical outputs while physical inputs drive game progression. The result is an arcade experience where players interact naturally through movement while spectators enjoy the synchronized light show.

Component	Platform	Purpose
Floor ESP32	MicroPython	Button polling, NeoPixel control, WiFi AP hosting
Ceiling ESP32	MicroPython	Disco patterns, stepper sequences, IR sensing, state machine
Bridge Server	Python 3 (IDLE Python)	HTTP proxy, button merging, command routing
Game Interface	HTML/JavaScript	Game logic, UI rendering, hardware sync hooks

2. System Logic

User Input Flow : User steps → Floor detects → Bridge polls → Merges buttons → Browser receives → Game logic → LED command → Floor updates → Visual feedback

Game State Flow (10 steps): Menu navigation → State command → Bridge forwards → Ceiling receives → State machine → Disco pattern change → User sees overhead effect

IR Sensor Flow (12 steps): Hand waves → IR triggers → Sticky flag → Stepper feedback → Bridge polls → Browser awards points → Celebration spin

Wrong Move Flow (9 steps): Wrong tile → Game detects → Flash red → Error event → Motors spin opposite → User sees/hears → Game continues

Technical Implementation Details

Timing Specifications

- • Bridge polling cycle: ~20ms (50Hz)
- • NeoPixel frame rate: 20ms (50fps)
- • Button debounce: 50ms window
- • IR sensor debounce: 500ms window
- • Button press to LED update: <50ms total latency
- • State change to disco pattern: <100ms
- • Stepper motor delay: 800-1200μs per step

Error Handling

- • Floor timeout: 1 second → Retry once, then fail gracefully
- • Ceiling timeout: 0.5 seconds → Return zeros, floor still works
- • WiFi disconnect: Floor auto-reconnects, bridge waits
- • Invalid command: Ignored silently, system continues
- • Button state conflict: Bridge uses most recent poll
- • NeoPixel buffer overflow: Clamped to array bounds

3. Flowchart

Visual representation of logic

USER ACTION
(Steps on Tile Button)

FLOOR ESP32 (MicroPython) • UI rendering
• GPIO detects button press (active LOW)
• Updates state array: buttons[4] = 1
• Debounce: 50ms window
• HTTP Server ready: 192.168.4.1:80

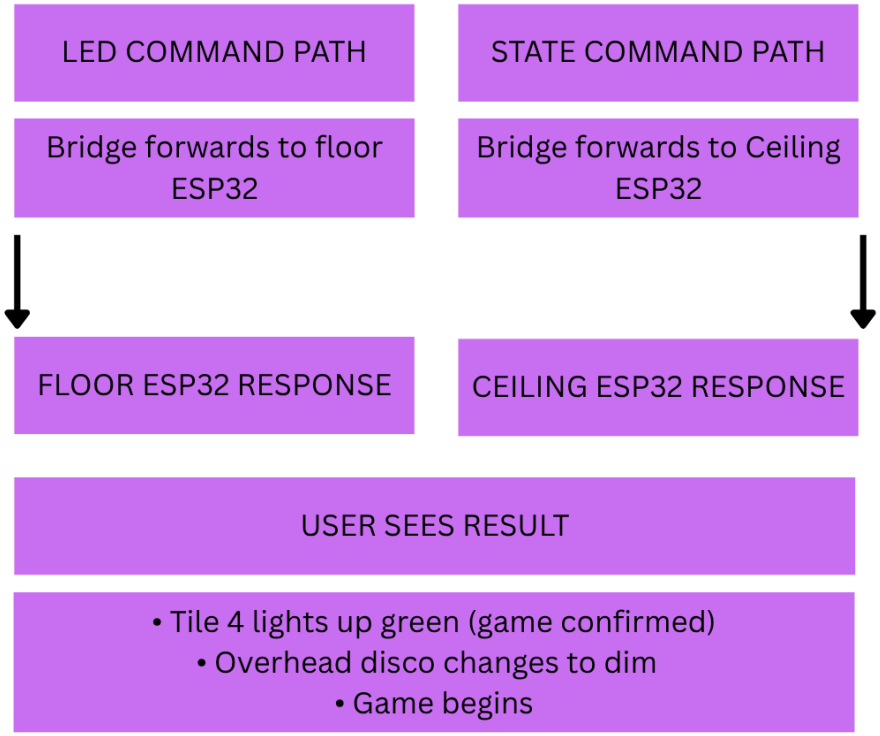
BRIDGE SERVER (Python)

Polling Loop (~20ms):

1. GET http://192.168.4.1/state
   Response: {"b":[0,0,0,0,1,0,0,0,0]}
2. GET http://192.168.4.100/state
   Response: {"b":[0,0]}
3. Merge: [0,0,0,0,1,0,0,0,0] + [0,0]
   Result: [0,0,0,0,1,0,0,0,0,0,0]

BROWSER (HTML/JavaScript)

• Polls bridge: GET http://localhost:8000/state
• Receives: {"b":[0,0,0,0,1,0,0,0,0,0,0]}
• Game logic: "Button 4 pressed = CENTER tile"
• Decision: Confirm game selection


LED COMMAND PATH
STATE COMMAND PATH
Bridge forwards to floor ESP32
Bridge forwards to Ceiling ESP32
FLOOR ESP32 RESPONSE
CEILING ESP32 RESPONSE
USER SEES RESULT
• Tile 4 lights up green (game confirmed)
• Overhead disco changes to dim
• Game begins